

EE332

Digital System Design

Dr. Qingfeng Zhang
Professor,
Department of Electronic and Electrical Engineering
Office: Room 240, South Tower of COE
Email: zhangqf@sustech.edu.cn
Web: emwave.ee.sustech.edu.cn

Chapter 9: Pipelined Design

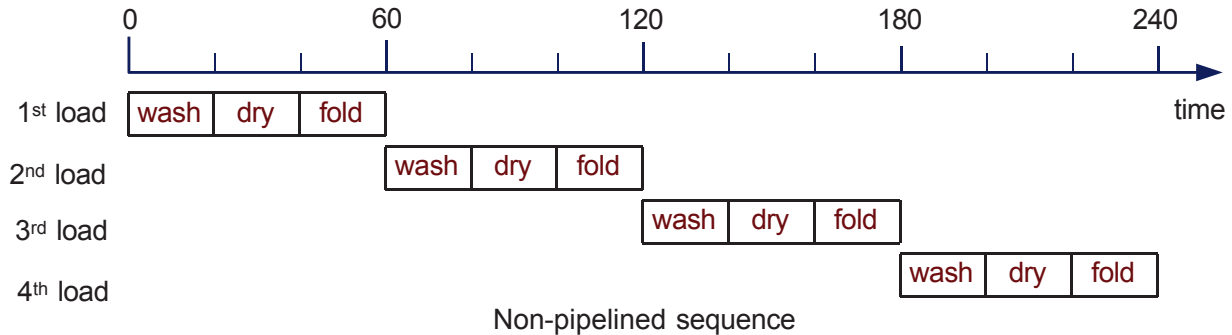
- Pipeline is an important technique to increase the performance of a system.
- The basic idea is to overlap the processing of several tasks so that more tasks can be completed in the same amount of time.

9.1 Delay and throughput

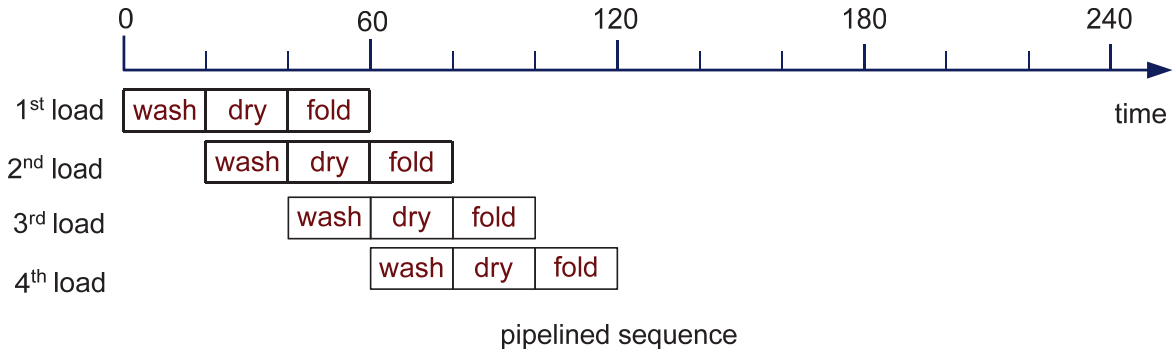
- Delay and throughput are the two criteria used to examine the performance of a system
 - Delay: the time required to complete one task.
 - Throughput: the number of tasks that can be completed per unit time.
- Adding pipeline to a combinational circuit can increase the throughput but *not* reduce the delay.

9.2 Overview on pipelined design

- The pipelining technique can be applied to a task that is processed in stages.
 - An example: Pipelined laundry.
 - A complete laundry includes the stages of washing, drying and folding, for example.



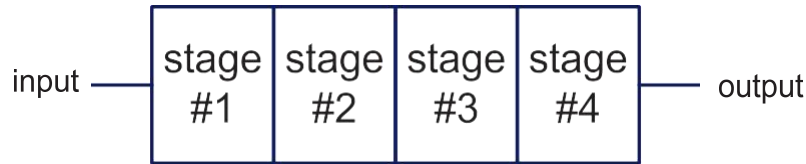
- For non-pipelined process, a new load cannot start until the previous load is completed.
 - It takes 240 minutes to complete the four loads.
 - The delay of processing one load is 60 minutes.
 - The throughput is $1/60$ load per minute.



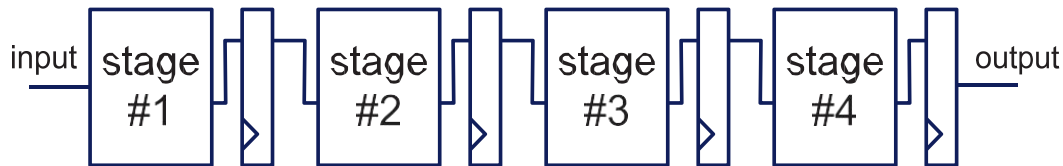
- For pipelined process,
 - It takes 120 minutes to complete the four loads.
 - The delay in processing one load **remains** 60 minutes.
 - The throughput increases to 2/60 load per minute.
 - To process k loads, it will take 40+20k minutes.
 - The throughput becomes $k/(40+20k)$ load per minute $\rightarrow$ 1/20 load per minute when k is large.

Pipelined combinational circuit

- A combinational circuit can be divided into stages so that the processing of different tasks can be overlapped.
- To ensure that the signals in each stage flow in the correct order and prevent any potential race, registers must be added between successive stages.
- The registers ensures that the signals can be passed to the next stage only at predetermined points.



Original combinational circuit



Pipelined circuit

Assume: propagation delay for each stage: T_1 , T_2 , T_3 and T_4 .

$$T_{\max} = \max(T_1, T_2, T_3, T_4);$$

Thus, the minimum clock period has to accommodate the longest delay plus the overhead introduced by the buffer register in each stage:

$$T_c = T_{\max} + T_r;$$

The effectiveness of the circuit

- Propagation delay:
 - non-pipelined circuit: $T_{\text{comb}} = T_1 + T_2 + T_3 + T_4$
 - pipelined circuit: $T_{\text{pipe}} = 4T_c = 4T_{\text{max}} + 4T_r$
- Throughput:
 - non-pipelined circuit: $1/T_{\text{comb}}$
 - pipelined circuit: $k/(3T_c + kT_c) \rightarrow 1/T_c$.

Ideally, for an N -stage circuit

- The propagation delay of each stage is identical (i.e.,
 $T_{\max} = T_{\text{comb}}/N$)
- The register overhead (T_r) is comparably small
 - $T_{\text{pipe}} = NT_c = NT_{\max} = T_{\text{comb}}$
 - Throughput: $1/T_c = 1/T_{\max} = N/T_{\text{comb}}$
- Ideally, it is desirable to have more stages in the pipeline. However, when N becomes large,
 - the propagation delay of each stage becomes smaller, but T_r remains the same; its effect cannot be ignored.
 - In reality, it is difficult to keep dividing the original combinational circuit into smaller and smaller stages.

9.3 Adding pipeline to a combinational circuit

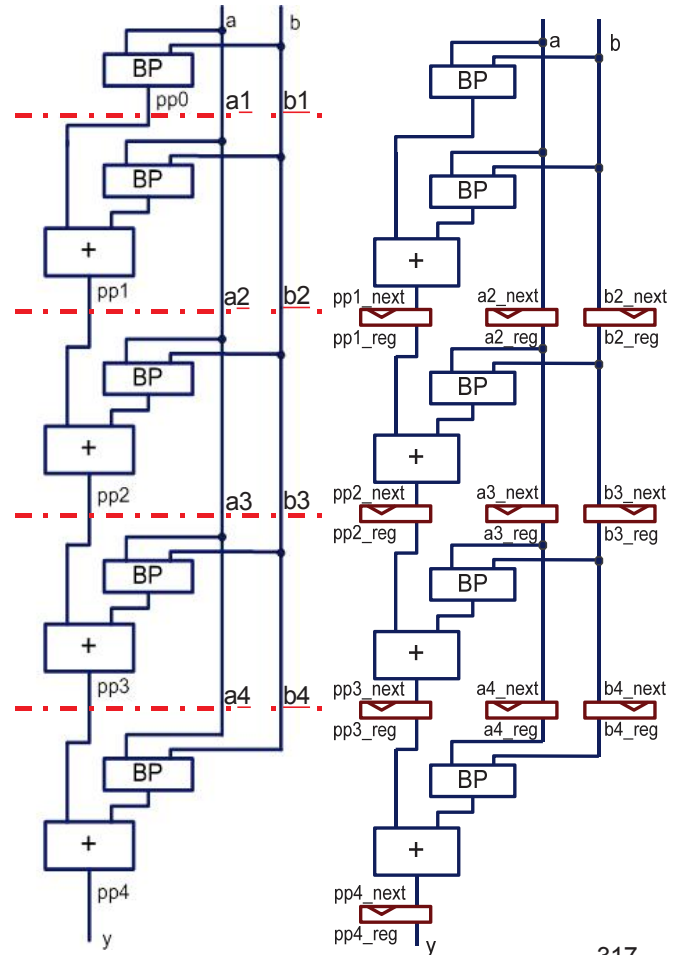
- The candidate circuits for effective pipeline design should include the following characteristics:
 - There is enough input data to feed the pipeline circuit.
 - The *throughput* is the main performance criterion.
 - The combinational circuit can be divided into stages with similar propagation delay.
 - The propagation delay of a stage is much longer than the delay incurred due to the register.

The procedure to convert a combinational circuit to a pipelined design

- Derive the block diagram of the original combinational circuit and arrange it as a cascading chain.
- Identify the major components and estimate the relative propagation delays of these components.
- Divide the chain into stages of similar propagation delays.
- identify the signals that cross the boundary of the chain.
- Insert registers for these signals in the boundary.

Example: Simple pipelined adder-based multiplier

x											a_4	a_3	a_2	a_1	a_0
											b_4	b_3	b_2	b_1	b_0
											a_4b_0	a_3b_0	a_2b_0	a_1b_0	a_0b_0
											a_4b_1	a_3b_1	a_2b_1	a_1b_1	a_0b_1
											a_4b_2	a_3b_2	a_2b_2	a_1b_2	a_0b_2
											a_4b_3	a_3b_3	a_2b_3	a_1b_3	a_0b_3
											a_4b_4	a_3b_4	a_2b_4	a_1b_4	a_0b_4
+															
y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0						



Non-pipelined multiplier in cascading stages

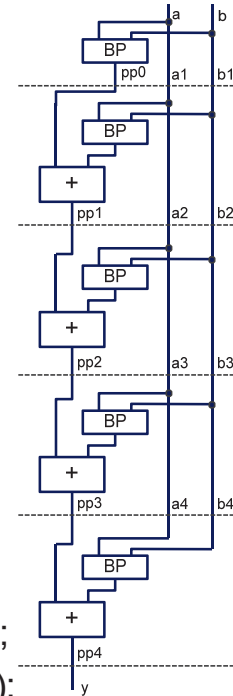
```

library IEEE;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity mult5 is
port (clk, reset : in std_logic;
      a, b : in std_logic_vector(4 downto 0);
      y: out std_logic_vector(9 downto 0));
end entity mult5;

architecture comb_arch of mult5 is
constant WIDTH : integer := 5;
signal a1, a2, a3, a4 : std_logic_vector (WIDTH-1 downto 0);
signal b1, b2, b3, b4 : std_logic_vector (WIDTH-1 downto 0);
signal bv0, bv1, bv2, bv3, bv4: std_logic_vector (WIDTH-1 downto 0);
signal bp0, bp1, bp2, bp3, bp4: std_logic_vector (2*WIDTH-1 downto 0);
signal pp0, pp1, pp2, pp3, pp4: std_logic_vector (2*WIDTH-1 downto 0);

```



begin

-- stage 0

bv0 <= (**others** => b(0));

bp0 <= "00000" & (**bv0** **and** a);

pp0 <= **bp0**;

a1 <= a;

b1 <= b;

-- stage 1

bv1 <= (**others** => b1(1));

bp1 <= "0000" & (**bv1** **and** a1) & "0";

pp1 <= **pp0** + **bp1**;

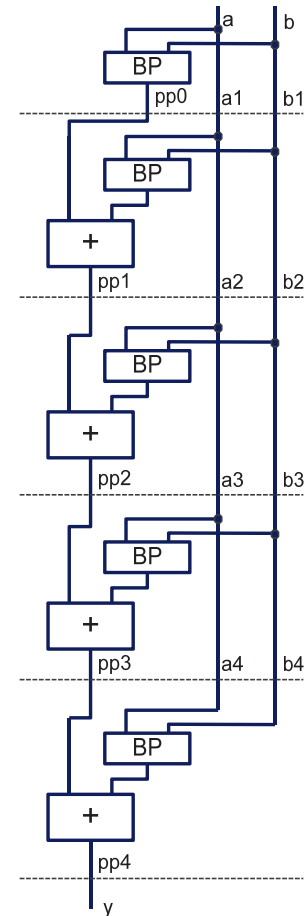
a2 <= a1;

b2 <= b1;

-- stage 2

bv2 <= (**others** => b2(2));

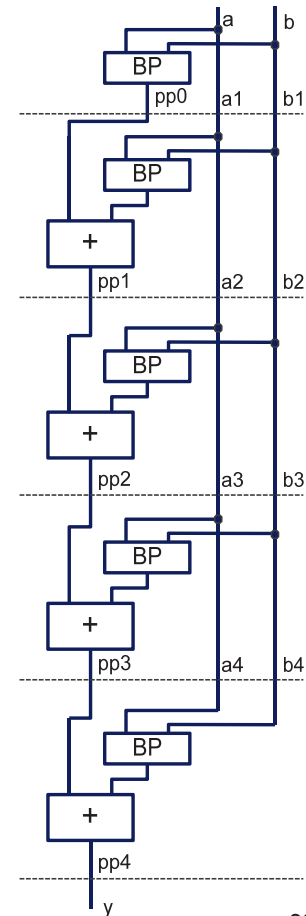
bp2 <= "000" & (**bv2** **and** a2) & "00";

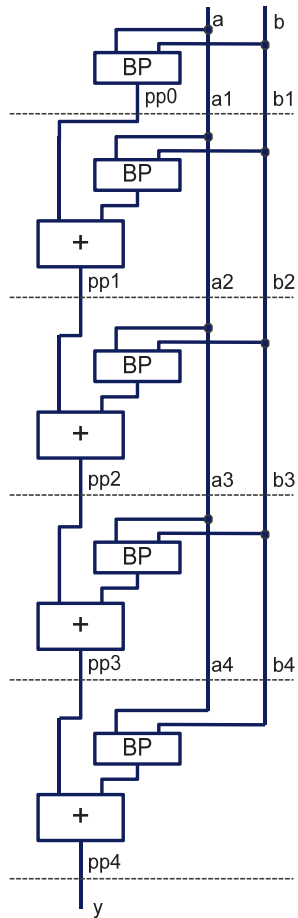


```

pp2 <= pp1 + bp2;
a3 <= a2;
b3 <= b2;
-- stage 3
bv3 <= (others => b3(3));
bp3 <= "00" & (bv3 and a3) & "000";
pp3 <= pp2 + bp3;
a4 <= a3;
b4 <= b3;
-- stage 4
bv4 <= (others => b4(4));
bp4 <= "0" & (bv4 and a4) & "0000";
pp4 <= pp3 + bp4;
-- output
y <= pp4;
end architecture comb_arch;

```



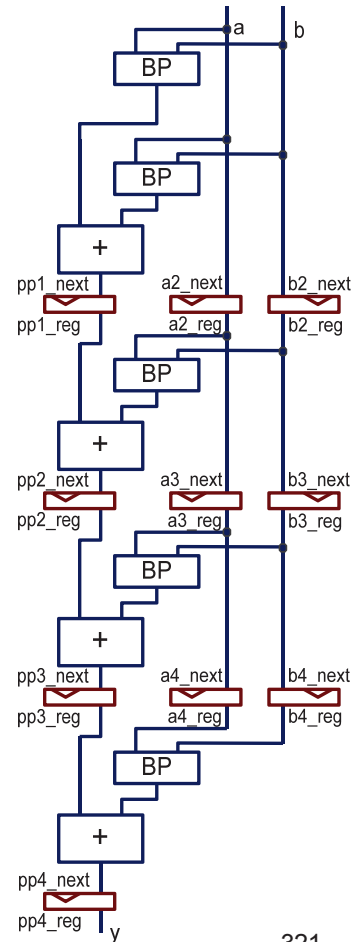


Non-pipelined circuit:

```
-- stage 2
pp2 <= pp1 + bp2;
-- stage 3
pp3 <= pp2 + bp3;
```

pipelined circuit:

```
-- register
if (reset = '1') then
    pp2_reg <= (others => '0');
    pp3_reg <= (others => '0');
elsif (clk'event and clk='1') then
    pp2_reg <= pp2_next;
    pp3_reg <= pp3_next;
end if;
...
-- stage 2
pp2_next <= pp1_reg + bp2;
-- stage 3
pp3_next <= pp2_reg + bp3;
```



Pipelined multiplier

architecture pipe_arch of mult5 is

constant WIDTH : integer := 5;

signal a2_reg, a3_reg, a4_reg,

b2_reg, b3_reg, b4_reg :

std_logic_vector (WIDTH-1 downto 0);

signal a1, a2_next, a3_next, a4_next:

std_logic_vector (WIDTH-1 downto 0);

signal b1, b2_next, b3_next, b4_next:

std_logic_vector (WIDTH-1 downto 0);

signal bv0, bv1, bv2, bv3, bv4:

std_logic_vector (WIDTH-1 downto 0);

signal bp0, bp1, bp2, bp3, bp4:

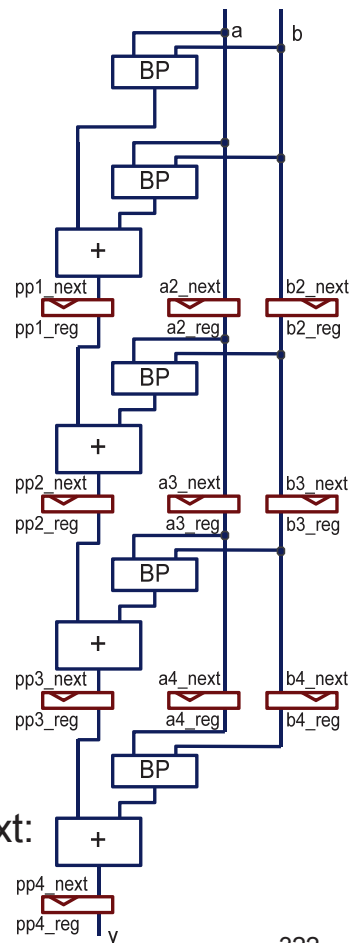
std_logic_vector (2*WIDTH-1 downto 0);

signal pp1_reg, pp2_reg, pp3_reg, pp4_reg:

std_logic_vector (2*WIDTH-1 downto 0);

signal pp0, pp1_next, pp2_next, pp3_next, pp4_next:

std_logic_vector (2*WIDTH-1 downto 0);



```
begin
```

```
-- pipeline registers
```

```
process (clk, reset)
```

```
begin
```

```
if (reset = '1') then
```

```
pp1_reg <= (others => '0');
```

```
pp2_reg <= (others => '0');
```

```
pp3_reg <= (others => '0');
```

```
pp4_reg <= (others => '0');
```

```
a2_reg <= (others => '0');
```

```
a3_reg <= (others => '0');
```

```
a4_reg <= (others => '0');
```

```
b2_reg <= (others => '0');
```

```
b3_reg <= (others => '0');
```

```
b4_reg <= (others => '0');
```

```
elsif (clk'event and clk = '1') then
```

```
pp1_reg <= pp1_next;
```

```
pp2_reg <= pp2_next;
```

```
pp3_reg <= pp3_next;
```

```
pp4_reg <= pp4_next;
```

```
a2_reg <= a2_next;
```

```
a3_reg <= a3_next;
```

```
a4_reg <= a4_next;
```

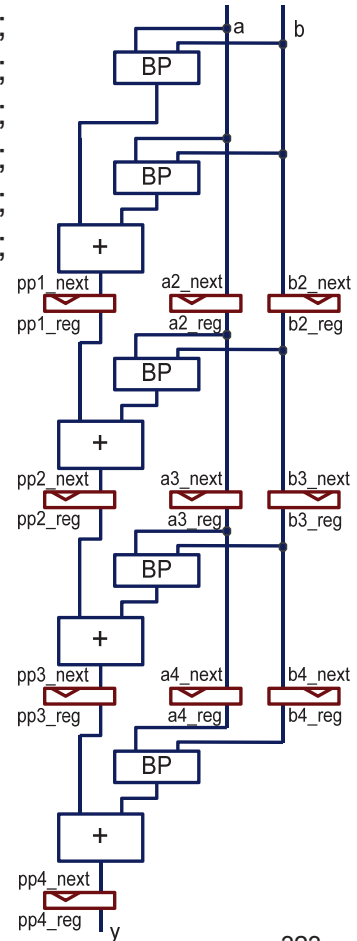
```
b2_reg <= b2_next;
```

```
b3_reg <= b3_next;
```

```
b4_reg <= b4_next;
```

```
end if;
```

```
end process;
```



-- merged stage 0 & 1 for pipeline

bv0 <= (**others** => b(0));

bp0 <= "00000" & (bv0 **and** a);

pp0 <= bp0;

a1 <= a;

b1 <= b;

--

bv1 <= (**others** => b1(1));

bp1 <= "0000" & (bv1 **and** a1) & "0";

pp1_next <= pp0 + bp1;

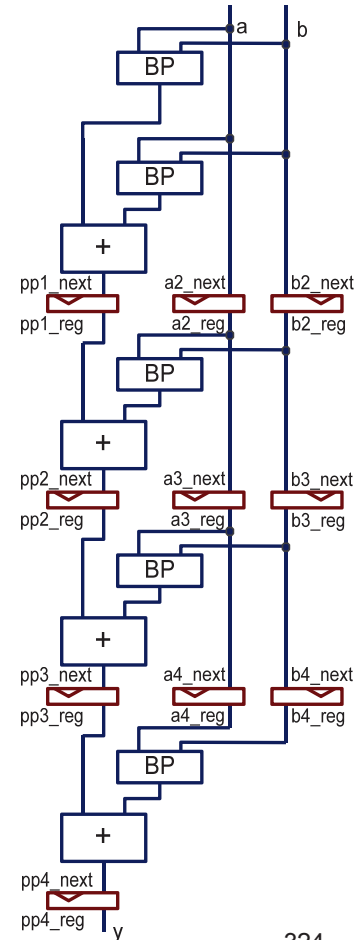
a2_next <= a1;

b2_next <= b1;

-- stage 2

bv2 <= (**others** => b2_reg(2));

bp2 <= "000" & (bv2 **and** a2_reg) & "00";



```
pp2_next <= pp1_reg + bp2;
```

```
a3_next <= a2_reg;
```

```
b3_next <= b2_reg;
```

```
-- stage 3
```

```
bv3 <= (others => b3_reg(3));
```

```
bp3 <= "00" & (bv3 and a3_reg) & "000";
```

```
pp3_next <= pp2_reg + bp3;
```

```
a4_next <= a3_reg;
```

```
b4_next <= b3_reg;
```

```
-- stage 4
```

```
bv4 <= (others => b4_reg(4));
```

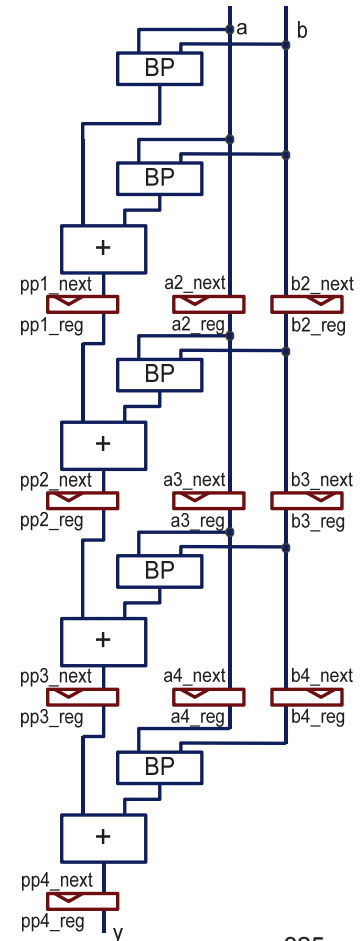
```
bp4 <= "0" & (bv4 and a4_reg) & "0000";
```

```
pp4_next <= pp3_reg + bp4;
```

```
-- output
```

```
y <= pp4_reg;
```

```
end architecture pipe_arch;
```



More efficient Pipelined multiplier

- Use a smaller $(n+1)$ -bit adder to replace the $2n$ -bit adder in an n -bit multiplier.
- Reduce the size of the partial-product register
- Reduce the size of the registers that hold the b signal.

architecture **efficient_pipe_arch** of **mult5** is

signal a2_reg, a3_reg, a4_reg: std_logic_vector(WIDTH-1 **downto** 0);

signal a1, a2_next, a3_next, a4_next:std_logic_vector(WIDTH-1 **downto** 0);

signal b1: std_logic_vector(4 **downto** 1);

signal b2_next, b2_reg: std_logic_vector (4 **downto** 2);

signal b3_next, b3_reg: std_logic_vector (4 **downto** 3);

signal b4_next, b4_reg: std_logic_vector (4 **downto** 4);

signal bv0, bv1, bv2, bv3, bv4:
std_logic_vector (4 **downto** 0);

signal bp0, bp1, bp2, bp3, bp4:
std_logic_vector (5 **downto** 0);

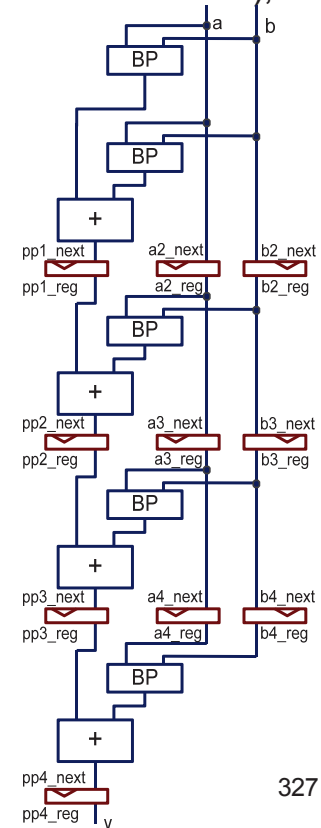
signal pp0: std_logic_vector (5 **downto** 0);

signal pp1_next, pp1_reg:std_logic_vector (6 **downto** 0);

signal pp2_next, pp2_reg: std_logic_vector (7 **downto** 0);

signal pp3_next, pp3_reg: std_logic_vector (8 **downto** 0);

signal pp4_next, pp4_reg: std_logic_vector (9 **downto** 0);



begin

-- pipeline registers

process (clk, reset)

begin

if (reset = '1') **then**

pp1_reg <= (**others** => '0');

pp2_reg <= (**others** => '0');

pp3_reg <= (**others** => '0');

pp4_reg <= (**others** => '0');

a2_reg <= (**others** => '0');

a3_reg <= (**others** => '0');

a4_reg <= (**others** => '0');

b2_reg <= (**others** => '0');

b3_reg <= (**others** => '0');

b4_reg <= (**others** => '0');

elsif (clk'event and clk = '1') **then**

pp1_reg <= pp1_next;

pp2_reg <= pp2_next;

pp3_reg <= pp3_next;

pp4_reg <= pp4_next;

a2_reg <= a2_next;

a3_reg <= a3_next;

a4_reg <= a4_next;

b2_reg <= b2_next;

b3_reg <= b3_next;

b4_reg <= b4_next;

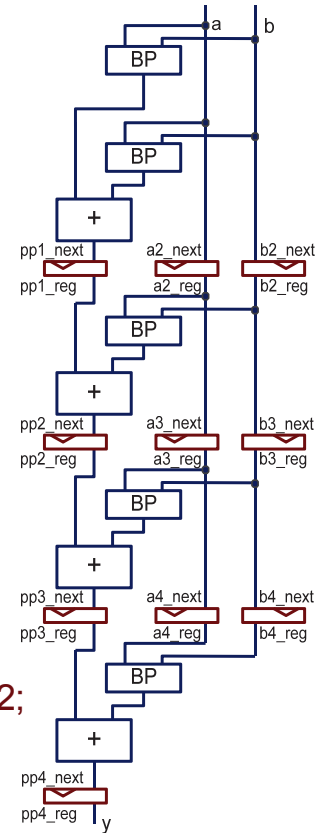
end if;

end process;


```

bv0 <= (others => b(0));
bp0 <= "0" & (bv0 and a);
pp0 <= bp0;
a1 <= a;
b1 <= b(4 downto 1);
--
bv1 <= (others => b1(1));
bp1 <= "0" & (bv1 and a1);
pp1_next(6 downto 1) <= ("0" & pp0(5 downto 1)) + bp1;
pp1_next(0) <= pp0(0);
a2_next <= a1;
b2_next <= b1(4 downto 2);
-- stage 2
bv2 <= (others => b2_reg(2));
bp2 <= "0" & (bv2 and a2_reg);
pp2_next(7 downto 2) <= ("0" & pp1_reg(6 downto 2))
pp2_next(1 downto 0) <= pp1_reg(1 downto 0);
a3_next <= a2_reg;
b3_next <= b2_reg(4 downto 3);

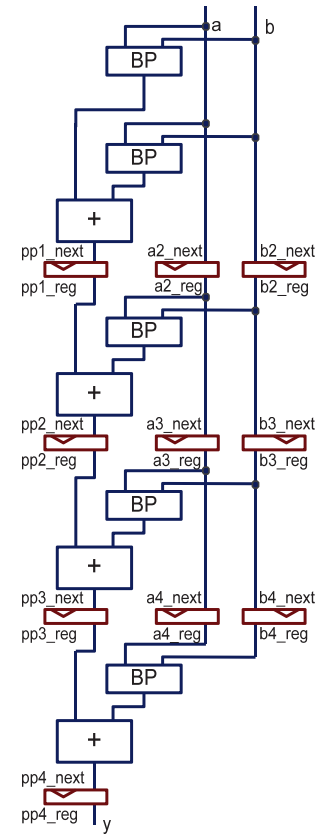
```



```

-- stage 3
bv3 <= (others => b3_reg(3));
bp3 <= "0" & (bv3 and a3_reg);
pp3_next(8 downto 3) <=
    ("0" & pp2_reg(7 downto 3)) + bp3;
pp3_next(2 downto 0) <= pp2_reg(2 downto 0);
a4_next <= a3_reg;
b4_next <= b3_reg(4);
-- stage 4
bv4 <= (others => b4_reg(4));
bp4 <= "0" & (bv4 and a4_reg);
pp4_next(9 downto 4) <=
    ("0" & pp3_reg(8 downto 4)) + bp4;
pp4_next(3 downto 0) <= pp3_reg(3 downto 0);
-- output
y <= pp4_reg;
end architecture efficient_pipe_arch;

```



Chapter 10. Subprograms, packages and libraries

- For complex design, VHDL provides mechanism for structuring programs, reusing modules
 - Subprograms such as functions and procedures for encapsulating commonly used operations
 - Packages and libraries for sharing large bodies of code.

10.1 Functions

- Functions are used to compute a value based on the values of the input parameters.
- Function declaration:

function func_name (parameter_list) **return** data_type;

formal parameter return value type

Parameter values in
functions are used,
but not changed
within the function.

- Structure of a function

```
function rising_edge (clock: std_logic) return boolean is  
  --  
  -- declarative region: declare variables local to the function  
  --  
  begin  
    -- body  
    -- sequential statement;  
    -- sequential statement;  
    -- --  
    return (value);  
end function rising_edge;
```

- Calling a function in a VHDL module:

rising_edge (enable); *-- positional association*



actual parameter,
the type must match to the formal parameter, i.e., enable
must be a **signal** of type std_logic.

rising_edge (clock => enable); *-- name association*

- Functions execute in zero simulation time.
 - Wait statement are not permitted in functions.

```

entity dff is
port (   D_in, CLK: in std_logic;
        D_out: out std_logic);
end entity dff;

architecture behavior of dff is
    function rising_edge(signal clock: in std_logic) return boolean is
    {
        variable edge : boolean :=FALSE;
        begin
            edge := (clock='1' and clock'event);
            return (edge);
        end function rising_edge;
    }
begin
    process (CLK) is
    begin
        if (rising_edge(CLK)) then
            D_out <= D_in;
        end if;
    end process;
end architecture behavior;

```

A majority function

- It returns '1' if two or more of the 3 input parameters, a , b and c are '1'.
- It can be treated as a shorthand for the $ab+ac+bc$ expression

```
architecture arch of ...
```

```
    -- declaration
```

```
    function maj(a, b, c: std_logic) return std_logic is
```

```
        variable result: std_logic;
```

```
    begin
```

```
        result := (a and b) or (a and c) or (b and c);
```

```
        return result;
```

```
    end function maj;
```

```
    signal i1, i2, i3, i4, x, y: std_logic;
```

```
begin
```

```
...
```

```
    x <= maj(i1, i2, i3) or i4;
```

```
    y <= i1 when maj(i2, i3, i4)='1' else ...
```

```
end arch;
```


Type conversion functions

- To make assignment from an object of one type to an object of another type.

– for example: `bit_vector` and `std_logic_vector`.

```
function to_bitvector (svalue: std_logic_vector) return bit_vector is  
    variable outvalue : bit_vector(svalue'length – 1 downto 0);  
begin  
    for i in svalue' range loop -- scan all elements of the array  
        case svalue(i) is  
            when '0' => outvalue (i) := '0';  
            when '1' => outvalue (i) := '1';  
            when others => outvalue (i) := '0';  
        end case;  
    end loop;  
    return outvalue;  
end function to_bitvector;
```

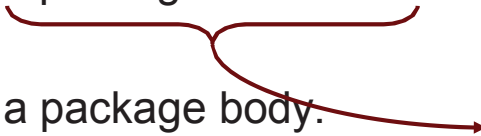
A Function performing $\lceil \log_2 n \rceil$

```
function log2c (n: integer) return integer is
  variable m, p: integer;
begin
  m := 0;
  p := 1;
  while p < n loop
    m := m+1;
    p := p*2;
  end loop;
  return m;
end function log2c;
```

Summary

- Unlike entity and architecture, functions (and procedures) are **not design units** and thus cannot be processed independently.
- In synthesis, functions should not be used to specify the design hierarchy, but should **be treated as a shorthand** for simple, repeatedly used operations.
- A function can be thought of as **an extension of the expression** and can be “called” whenever an expression is used.

9.2 Package

- The primary purpose of a package is to collect elements that can be shared (globally) among two or more design units. A package is represented by:
 - a package declaration and, optionally,
 - a package body.

contains a set of declarations that may possibly be shared by many design units. It defines items that can be made visible to other design units, for example, a function declaration.
- Package declaration and package body are design units of VHDL.

An example of a package declaration

```
package SYNTH_PACK is
  constant LOW2HIGH : TIME := 20ns;
  type ALU_OP is (ADD, SUB, MUL, DIV, EQL);
  type MVL is ('U', '0', '1', 'Z');
  component NAND2
    port (A, B : in MVL; C : out MVL);
  end component;
  -- subprogram, type, constant, signal, variable, component ...,
  -- and use clause can be declared in package declaration
end package SYNTH_PACK;
```

- If the declarations include items such as functions or procedure declarations, the behavior of the function and procedure are specified in a separate design unit called the package body.

An example of a package body

```
use WORK.SYNTH_PACK.all;
```

```
package PROGRAM_PACK is  
  constant PROP_DELAY : TIME;  
  function ISZERO(A: MVL) return boolean;  
end package PROGRAM_PACK;
```

} Package
declaration

```
package body PROGRAM_PACK is  
  constant PROP_DELAY : TIME := 15ns;  
  
  function ISZERO(A: MVL) return boolean is  
  begin  
    if (A='0') return TRUE;  
    else return FALSE;  
    end if;  
  end function ISZERO;  
end package body PROGRAM_PACK;
```

package body name:
must be the same as
of its corresponding
package declaration.

} Package
body

Note:

- An item declared inside a package body has its scope restricted to be within the package body, and this item cannot be made visible in other design unit. (This is in contrast to items declared in a package declaration).

10.4 Design libraries

- Each design unit - entity, architecture, configuration, package declaration, package body is analyzed (compiled) and placed in a design library for subsequent use.
- To use a design library, the library must be declared by its logical name.

library logical-library-name1, logical-library-name2,...;

- In VHDL, the libraries STD and WORK are implicitly declared.



standard packages
provided with the
VHDL distributions

working directory

- Once a library is declared, all of the functions, procedures, and type declaration of a package in that library can be made accessible to a VHDL model through the **use** clause.

library IEEE

use IEEE.std_logic_1164.**all**;

- These clauses apply only to the immediate entity-architecture pair! ***Visibility must be established for other design units separately!***